PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ - PUCPR
FUNDAMENTOS DE SISTEMAS CIBERFÍSICOS

FELIPE GABRIEL LIMA
GIOVANNI MARTINS VILLAR TORINO
KAUAN BRAGA MAGRI
LUIZ FELIPE PERANTONI PEREIRA
PEDRO HENRIQUE PRASIDOS DA SILVA

SEMÁFORO INTELIGENTE DE BOTOEIRA COM CONTROLE DE ACIONAMENTO POR MEIO DE SENSOR DE PRESENÇA

TOLEDO
1º SEMESTRE DE 2026

RESUMO

A mobilidade urbana no Brasil encontra-se em cenário crítico, marcado pelo aumento do uso de transporte individual e pela redução de cerca de 30% na demanda por transporte público nas últimas décadas (CARVALHO et al., 2023), com reflexos diretos nos elevados índices de congestionamento das principais cidades brasileiras (TOMTOM, 2025). Nesse contexto, o conceito de cidades inteligentes apresenta-se como caminho para soluções que promovam maior fluidez e qualidade de vida urbana. O presente trabalho aborda o desenvolvimento e a implementação de um protótipo funcional de semáforo inteligente com botoeira e controle de acionamento por sensor de presença ultrassônico (HC-SR04), construído sobre a plataforma Arduino Uno. O sistema visa eliminar interrupções desnecessárias do tráfego de veículos, validando a intenção de travessia do pedestre por meio da detecção constante na calçada antes de efetivamente fechar o sinal para os veículos. A arquitetura de hardware implementada baseia-se em um Módulo Semáforo LED para sinalização veicular, um display LCD 16x2 com módulo I2C para comunicação visual, um botão (push button) e o sensor ultrassônico. Os resultados dos testes demonstraram a eficácia do sistema através de uma lógica de programação que garante transições fluidas, operação contínua e a comprovação conceitual de que soluções de baixo custo são viáveis para otimizar a segurança e o fluxo em cruzamentos urbanos.

Palavras-chave: mobilidade urbana; sinalização semafórica; congestionamento; cidades inteligentes;

1 INTRODUÇÃO

1.1 Mobilidade Urbana no Brasil

Anteriormente ao aprofundamento no projeto proposto de sinalização semafórica inteligente, é necessário entender o cenário do Brasil em relação à mobilidade urbana, para elucidar os fatores que influenciaram na escolha do projeto.

Primeiramente, mostra-se necessário entender, por lei, o que se define como transporte urbano e, além disso, propriamente a mobilidade urbana. Segundo a Lei nº 12.587 (BRASIL, 2012), o transporte urbano pode ser definido como um grupo das formas de transportes, sejam eles públicos ou privados, que são aproveitados para proporcionar o deslocamento de pessoas e cargas em cidades.

Por sua vez, a mobilidade urbana é definida como o cenário em que é realizado esse deslocamento de pessoas e cargas (BRASIL, 2012).

Em relação à discussão sobre mobilidade urbana no Brasil, de acordo com Carvalho et al. (2023), diversos estudos elucidam um cenário de aumento no uso de automóveis e motocicletas e redução do uso de transportes públicos pela sociedade brasileira.

Além disso, segundo Carvalho et al. (2023), nas últimas décadas, estima-se que os transportes públicos coletivos tiveram perda de demanda em cerca de 30% no Brasil. No entanto, há consequências significativas com esse cenário, como aumento de sinistros de trânsito (acidentes), da poluição e de tempo de viagens, impactando negativamente na mobilidade urbana (CARVALHO et al., 2023).

1.2 Indicadores Sobre Mobilidade Urbana no Brasil

Com base em todo o cenário abordado, mostra-se fundamental apresentar dados reais e atualizados acerca dos índices de congestionamento e tempo gasto neles, no Brasil. A partir de dados do TomTom (2025), é possível identificar que diversas cidades brasileiras estão sofrendo aumentos nas taxas de congestionamento médio em relação a 2024, com diversas delas superando 50%, como Recife (64,7%), Porto Alegre (59,6%), Belo Horizonte (58,6%), São Paulo (58,5%), Rio de Janeiro (57,8%), Fortaleza (56,9%) e Curitiba (54%).

Nesse sentido, outra informação essencial para entender o contexto do tráfego no Brasil, baseia-se no tempo perdido anualmente durante os horários de pico, valor mensurado considerando o tempo extra perdido nos horários de pico, devido ao congestionamento em um

percurso de 20 km diários, apenas em dias úteis. De acordo com dados do TomTom (2025), Curitiba é a cidade com maior quantidade de tempo perdido em congestionamentos (135 horas por ano), seguida de São Paulo (132 horas por ano), Recife e Belo Horizonte (ambas com 130 horas por ano), Porto Alegre (125 horas por ano) e Fortaleza (121 horas por ano), considerando apenas as cidades com mais de 120 horas por ano.

Todo o contexto abordado acerca das taxas de congestionamento médio brasileiro, traz um alerta em relação à mobilidade urbana no país, visto que, segundo TomTom (2025), Recife está em 15º lugar nas cidades com maior congestionamento médio do mundo.

1.3 Cidades Inteligentes e Sustentáveis

Entende-se que o conceito de Cidades Inteligentes é bastante amplo, o que impacta diretamente na quantidade de definições possíveis para o termo. Nota-se que, com base no que aborda Ferreira (2021), em grande parte, as Cidades Inteligentes estão relacionadas com o uso de TICs (Tecnologia da Informação e Comunicação).

Assim, segundo Ferreira (2021), em meio às diversas definições, é possível obter um certo padrão, focando na relação dos TICs com impacto na sustentabilidade do desenvolvimento das áreas urbanas, na qualidade de vida, na sociedade e na governança.

Por haver certa relação com sustentabilidade, conclui-se que podemos definir o conceito de Cidades Inteligentes e Sustentáveis como: tipo de cidades que utilizam produtos da área de Tecnologia da Informação e Comunicação de forma alinhada com flexibilidade, inovação, segurança e com possibilidade de expansão. Essas cidades visam aprimorar a qualidade de vida, a saúde, a educação, a segurança, o meio ambiente e a própria gestão da cidade (FERREIRA, 2021).

1.4 Discussão e Justificativa Técnica

A partir de toda a abordagem realizada em relação ao cenário de mobilidade urbana, cidades inteligentes e dados atualizados sobre tráfego no Brasil, é possível concluir que o país necessita de soluções alinhadas às cidades inteligentes, que auxiliem na redução do congestionamento do tráfego e que alcance uma mobilidade urbana eficiente e segura.

Assim, o presente trabalho mostra-se alinhado com essa necessidade, propondo um protótipo de sistema de sinalização semafórica inteligente capaz de otimizar o fluxo de

veículos e pedestres, promovendo maior mobilidade urbana e, consequentemente, mais fluidez e segurança no trânsito.

1.5 Escopo Abordado

O escopo deste projeto abrange o planejamento e o desenvolvimento de um protótipo físico de semáforo inteligente com botoeira e sensor de presença, construído sobre a plataforma Arduino Uno. Além disso, mostra-se fundamental citar os cenários que não estão incluídos no escopo deste projeto, como a conectividade com redes externas ou sistemas de tráfego municipais, o monitoramento remoto, a integração com outros semáforos e qualquer forma de implantação em via pública real. O projeto limita-se ao desenvolvimento e validação do protótipo em ambiente acadêmico/laboratorial, buscando comprovar conceitualmente e com protótipo prático o alinhamento do projeto aos princípios das cidades inteligentes e sustentáveis.

2 OBJETIVOS

O presente projeto tem como objetivo geral desenvolver um protótipo de sistema de sinalização semafórica inteligente, controlado por sensor de presença e acionado por botoeira, buscando otimizar o fluxo de veículos e pedestres em cruzamentos urbanos, promovendo maior segurança, fluidez e acessibilidade no trânsito.

Para alcançar este objetivo principal, foram definidos os seguintes objetivos específicos:

- Melhorar o fluxo do tráfego de veículos, evitando interrupções desnecessárias do trânsito, de forma que o sistema só interrompa a circulação quando houver de fato um pedestre aguardando na calçada;
- Implementar um sistema semafórico com lógica adaptativa, capaz de detectar a presença de pedestres na região da calçada próxima ao semáforo, por meio do sensor Ultrassônico HC-SR04 e ajustar o ciclo de sinalização de forma autônoma;
- Implementar uma interface de comunicação visual por meio de display LCD 16x2, fornecendo informações em tempo real aos pedestres, incluindo mensagens de estado e contagem regressiva;

- Garantir a segurança operacional do sistema por meio de requisitos funcionais e não funcionais bem definidos;
- Desenvolver um protótipo físico funcional utilizando Arduino Uno como unidade de controle central, integrando os componentes eletrônicos necessários (Módulo Semáforo LED, botão, sensor Ultrassônico e display LCD) de forma organizada e devidamente documentada;
- Contribuir com o contexto de cidades inteligentes e sustentáveis, propondo uma solução de baixo custo e replicável alinhada às demandas de mobilidade urbana identificadas no cenário brasileiro.

3 REQUISITOS FUNCIONAIS FINAIS

Segue abaixo uma tabela comparativa entre os requisitos funcionais propostos e os implementados de forma consolidada no projeto:

ID	Requisito	Situação	Observação
RF1	O sistema deve operar continuamente no ciclo verde, amarelo e vermelho para os automóveis	**APLICADO COM RESSALVAS**	O sistema opera de forma que o semáforo permanece verde para os automóveis até um pedestre ser identificado pelo sensor e pressionar o botão. A partir disso, o semáforo altera para amarelo e depois para vermelho, retornando por fim ao verde.
RF2	O semáforo (Leds) vermelho para carros deve estar ativado antes do display LCD exibir a	**APLICADO**	O sistema exibe o sinal vermelho para os carros e depois de um tempo de segurança a mensagem para os pedestres atravessarem

	mensagem “Vez dos pedestres” para os pedestres, sem nenhuma sobreposição.		(“PEDESTRES”) é exibida no LCD.
RF3	O display LCD deve exibir "Vez dos veículos" com contagem regressiva durante o verde.	**APLICADO**	O sistema exibe o sinal verde para os carros, a mensagem no LCD (“VEICULOS”) junto de um temporizador até um pedestre pressionar o botão.
RF4	Em caso de falha ou reset, o sistema deve iniciar com semáforo (LEDs) vermelho ativo para os automóveis e o display LCD exibindo a mensagem “Aguarde...” para os pedestres.	**NÃO APLICADO**	Após o botão reset ser pressionado o LCD reinicia e exibe a mensagem “VEICULOS”, o temporizador e o sinal verde para os automóveis.
RF5	O sistema jamais deve liberar carros e pedestres ao mesmo tempo.	**APLICADO**	O sistema de forma alguma irá abrir o semáforo para os automóveis sendo que pedestres estão atravessando.
RF6	O display LCD deve exibir "ATENÇÃO!" com contagem regressiva durante o amarelo.	**APLICADO**	O sistema exibe uma mensagem de qual o significado do led amarelo presente no semáforo.

RF7	O sistema deve registrar o acionamento do botão pelo pedestre.	**APLICADO**	O botão após ser pressionado é registrado pelo sistema, após o temporizador dos automóveis terminar o LCD irá sinalizar que o pedestre pode cruzar.
RF8	O display LCD deve exibir "AGUARDE..." com contagem regressiva até o ciclo dos automóveis terminar.	**APLICADO PARCIALMENTE**	O display exibe o temporizador dos automóveis, junto da mensagem “VEICULOS” sinalizando que o semáforo está aberto para os veículos
RF9	Pressionar o botão múltiplas vezes não deve reiniciar nem acumular solicitações.	**APLICADO**	Ao pressionar o botão várias vezes, o sistema mantém apenas a primeira solicitação ativa, evitando reinícios e múltiplas execuções ao mesmo tempo.
RF10	O botão deve ser ignorado caso o sinal de pedestres já esteja ativo.	**APLICADO**	O sistema ignora o sinal vindo do botão caso o semáforo já esteja fechado para os automóveis.
RF11	O sinal deve abrir para pedestres somente após o vermelho para carros estar ativo.	**APLICADO**	O LCD só deve sinalizar que é a vez dos pedestres após o semáforo estar fechado para os automóveis.
RF12	O display LCD deve exibir "Vez dos pedestres" com	**APLICADO**	O LCD exibe uma contagem regressiva durante a passagem de pedestres.

	contagem regressiva durante a travessia.		
RF13	O sensor HC-SR04 (ultrassônico) deve monitorar a presença do pedestre na calçada próxima ao semáforo durante a travessia.	**APLICADO**	O sensor identifica a presença do pedestre para garantir que ele ainda está no local após o pressionamento do botão.
RF14	Se o PIR detectar que a calçada foi liberada antes do tempo, o sistema deve iniciar o ciclo de liberação antecipado.	**APLICADO PARCIALMENTE**	O sistema verifica a ausência do pedestre via HC-SR04, porém a liberação antecipada depende de um tempo mínimo fixo antes de ser acionada.
RF15	O display LCD deve exibir "LIBERANDO..." com contagem regressiva curta como margem de segurança.	**NÃO APLICADO**	O sistema alterna de “VEICULOS” para “ATENCAO” e ek seguida para “PEDESTRES”.
RF16	Após o ciclo de liberação, o sistema deve retornar automaticamente ao ciclo dos carros.	**APLICADO**	Após o término do temporizador dos pedestres o sitema retorna a permanecer o sinal aberto para os automóveis até que outro pedestre pressione o nbotão solicitando a travessia.

RF17	O LED verde deve acender somente quando for a vez dos carros.	**APLICADO**	
RF18	O LED amarelo deve acender como aviso antes de fechar o sinal para os carros.	**APLICADO**	
RF19	O LED vermelho deve acender antes de qualquer abertura para pedestres.	**APLICADO**	
RF20	Os três LEDs nunca devem acender ao mesmo tempo.	**APLICADO**	
RF21	O ciclo "LIBERANDO..." deve existir como margem de segurança antes de reabrir o sinal para os carros.	**NÃO APLICADO**	
RF22	A contagem regressiva no display LCD deve ser atualizada a cada segundo.	**APLICADO**	
RF23	O sistema deve ignorar o sinal do	**NÃO APLICADO**	Sensor PIR não foi utilizado

	PIR enquanto não for a vez dos pedestres.		
RF24	O sensor PIR deve aguardar um tempo mínimo antes de considerar que a calçada próxima ao semáforo está livre.	**NÃO APLICADO**	Sensor PIR não foi utilizado
RF25	O sistema deve detectar quando o pedestre ainda está na calçada e manter o sinal fechado para os veículos até ele terminar de atravessar.	**APLICADO PARCIALMENTE**	O sistema identifica se o pedestre está aguardando na calçada ao fim de cada ciclo do estado vermelho do semáforo, no entanto, não possui controle de manter o semáforo fechado até o pedestre atravessar, mas somente um tempo fixo.
RF26	O botão deve funcionar como solicitação de travessia, não como comando imediato.	**APLICADO**	
RF27	O display LCD deve limpar a tela ao trocar de mensagem, sem deixar texto anterior visível.	**APLICADO**	
RF28	O sistema deve registrar se o	**NÃO APLICADO**	

	pedestre atravessou ou saiu do local sem atravessar.		
RF29	Caso o PIR não detecte presença logo após o botão ser pressionado, o sistema deve ignorar o acionamento e manter o ciclo normal dos veículos.	**NÃO APLICADO**	Sensor PIR não foi utilizado
RF30	O ciclo de amarelo deve ter duração fixa independente de qualquer acionamento externo.	**APLICADO**	

4 REQUISITOS NÃO FUNCIONAIS FINAIS

Segue abaixo uma tabela comparativa entre os requisitos não funcionais propostos e os implementados de forma consolidada no projeto:

ID	Requisito	Situação	Observação
RNF1	O botão deve ser reconhecido em no máximo 500ms.	**APLICADO**	

RNF2	O sensor PIR deve ser lido o tempo todo sem causar atrasos no ciclo principal.	**NÃO APLICADO**	Sensor PIR não foi usado no projeto final
RNF3	O sistema deve funcionar de forma contínua e sozinho, sem precisar de ajuda externa.	**APLICADO**	
RNF4	O display LCD deve estar posicionado acima do botão na calçada, voltado ao pedestre.	**NÃO APLICADO**	O display foi implementado ao lado do botão
RNF5	As mensagens do display LCD devem ser curtas, claras e de fácil leitura em qualquer momento do ciclo.	**APLICADO**	
RNF6	O código deve ser dividido em funções separadas, cada uma com uma responsabilidade clara.	**APLICADO**	
RNF7	O sistema deve ser capaz de rodar sem interrupções por longos períodos sem	**APLICADO**	

	travar ou reiniciar sozinho.		
RNF8	O display LCD deve ter contraste suficiente para ser legível tanto de dia quanto à noite.	**APLICADO**	
RNF9	O tempo de travessia dos pedestres deve ser longo o suficiente para pessoas com mobilidade reduzida atravessarem.	**APLICADO PARCIALMENTE**	Para o protótipo, o tempo é fixo em 10 segundos para todos pedestres. Em uma versão melhorada, poderia ser utilizado módulos como leitor RFID, onde pessoas com mobilidade reduzida poderiam ter mais tempo para a travessia.
RNF10	Os resistores devem ser calculados corretamente para que os LEDs operem dentro da tensão e corrente adequadas.	**APLICADO**	
RNF11	O sensor PIR deve estar posicionado de forma a cobrir toda a área da calçada próxima ao semáforo de pedestres.	**APLICADO**	
RNF12	O botão deve estar em altura acessível,	**NÃO APLICADO**	Protótipo final foi feito em maquete, impossibilitando esse tipo de especificação

	inclusive para cadeirantes.		
RNF13	O sistema deve consumir energia de forma eficiente, sem sobrecarregar o microcontrolador.	**APLICADO**	
RNF14	O código deve ser escrito de forma legível, com comentários explicando cada parte.	**NÃO APLICADO**	O código foi escrito sem comentários
RNF15	Qualquer pessoa com conhecimento básico em Arduino deve conseguir entender e ajustar o código.	**APLICADO**	Código foi realizado de forma simples e didática
RNF16	O sistema deve ser montado de forma que os componentes possam ser substituídos individualmente em caso de defeito.	**APLICADO**	
RNF17	O tempo mínimo do ciclo verde dos carros deve ser respeitado mesmo se	**APLICADO**	

	o botão for pressionado logo no início.		
RNF18	A transição entre estados deve ser suave, sem pulos ou comportamentos inesperados no display LCD ou nos LEDs.	**APLICADO**	

5 ARQUITETURA IMPLEMENTADA

A arquitetura final do projeto baseia-se em componentes essenciais para a realização do fluxo, com responsabilidades bem divididas entre eles, buscando alcançar o funcionamento final esperado. Para o processamento geral foi utilizado o Arduíno UNO, que é o componente principal do projeto, responsável por interligar todos os elementos do sistema, realizando a verificação da interação com os sensores e o controle de toda a lógica do sistema. Foi utilizado a linguagem C/C++ na programação do sistema.

Além disso, ao contrário da proposta anterior em utilizar o Sensor PIR HC-SR501 para detecção de presença, optou-se por utilizar o Sensor Ultrassônico HC-SR04, visto que proporcionou um melhor funcionamento do sistema em geral.

O sensor ultrassônico HC-SR04 funciona com ondas sonoras. Ele emite um pulso ultrassônico e, caso haja um objeto à frente, essa onda retorna ao sensor. A partir desse retorno, é possível calcular a distância com base na velocidade do som.

O sensor possui dois pinos importantes: Trig e Echo. O pino Trig é responsável por enviar o pulso, enquanto o pino Echo é responsável por receber o retorno desse sinal e informar ao Arduino o tempo que a onda levou para voltar. Com base nessa informação, o Arduino calcula a distância do objeto que está dentro do alcance do sensor ultrassônico.

Em relação a solicitação de travessia, foi utilizado um botão (push button), funcionando como uma chave que abre ou fecha um circuito elétrico, convertendo uma ação mecânica em um sinal elétrico.

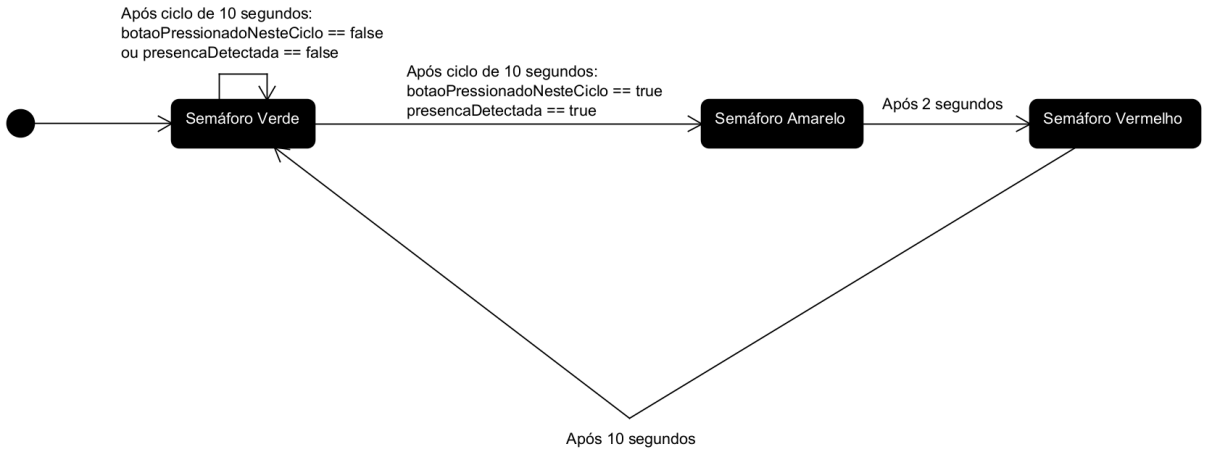
Para a comunicação visual com o pedestre, utilizamos um display LCD 16x2, que possui 16 colunas e 2 linhas. Com esse LCD, utilizamos um módulo chamado I2C, que reduz significativamente a quantidade de cabos necessários para ligar o display ao Arduino. Com esse módulo, o circuito fica mais organizado e sobram mais pinos disponíveis para os demais componentes do projeto.

Para representar o semáforo, ao contrário da ideia inicial de utilizar LEDs separados, foi optado por utilizar um Módulo Semáforo LED. O controle desse módulo é idêntico ao controle de LEDs normais, não sendo necessário conhecimento extra para utilizá-los. No sistema, os LEDs ajudam a mostrar o estado do semáforo, indicando quando os veículos devem seguir ou parar. Além disso, os LEDs são simples de utilizar com Arduino, possuem baixo consumo de energia e são adequados para representar sinais luminosos em maquetes e protótipos. Para evitar danos, normalmente são utilizados junto com resistores, que limitam a corrente elétrica que passa pelo componente.

Por fim, para ligar os componentes de forma mais organizada, foi utilizada uma Protoboard, responsável pela montagem do circuito sem a necessidade de solda. Também podem ser utilizadas placas perfuradas, caso seja desejada uma montagem mais fixa e profissional.

6 DIAGRAMA DE ESTADOS

Buscando proporcionar um melhor entendimento geral sobre o fluxo de estados do projeto, foi desenvolvido um Diagrama de Estados, indicando os estados principais e os eventos de transição entre eles.



Detalhamento: o sistema sempre inicia no estado verde e, a cada final do ciclo de 10 segundos (valor utilizado para o protótipo, onde em contexto real seria maior) é verificado se o botão foi acionado e há presença detectada. Caso a resposta seja verdadeira para ambas condições, o estado do semáforo muda para amarelo, fica 2 segundos neste estado e o estado muda para o vermelho (disponível para os pedestres atravessarem), onde após 10 segundos retornará para o verde, marcando o início de um novo ciclo. No entanto, caso uma ou ambas condições não sejam atendidas, simplesmente é reiniciado o ciclo, se mantendo no estado verde, garantindo o fluxo esperado.

7 IMPLEMENTAÇÃO

7.1 Código-Fonte

```
#include <LiquidCrystal_I2C.h>

#define greenLed 4
#define yellowLed 5
#define redLed 6
#define buttonPin 2
#define triggerPin 12
#define echoPin 13

unsigned long duracaoVerde = 10000;
unsigned long duracaoAmarelo = 2000;
unsigned long duracaoVermelho = 10000;

float distanciaLimiteCM = 15;
long duracaoPulsoEcho;
float distanciaCM;

int estadoConfirmadoBotao = HIGH;
int ultimoEstadoBotao = HIGH;
unsigned long momentoUltimaMudancaBotao = 0;
unsigned long duracaoDebounce = 50;

int estadoVerde = 0;
int estadoAmarelo = 1;
int estadoVermelho = 2;

int estadoAtualSemaforo;

unsigned long momentoInicioEstado = 0;

boolean botaoPressionadoNesteCiclo;

boolean presencaDetectada;

unsigned long momentoUltimaAtualizacaoLCD = 0;
unsigned long duracaoAtualizacaoLCD = 1000;

unsigned long momentoUltimaLeituraUltrassonico = 0;
unsigned long duracaoLeituraUltrassonico = 60;

LiquidCrystal_I2C myLcd(0x27, 16, 2);
```

```
void setup() {
  Serial.begin(9600);

  pinMode(greenLed, OUTPUT);
  pinMode(yellowLed, OUTPUT);
  pinMode(redLed, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
  pinMode(triggerPin, OUTPUT);
  pinMode(echoPin, INPUT);

  myLcd.init();
  myLcd.backlight();
  myLcd.clear();

  botaoPressionadoNesteCiclo = false;
  presencaDetectada = false;

  estadoAtualSemaforo = 0;

  momentoInicioEstado = millis();
  ligarVerde();
}


void loop() {
  lerBotao();
  lerUltrassonico();
  controlarSemaforo();
  mostrarContagemRegressiva();
}

void lerBotao() {
  int estadoAtualBotao = digitalRead(buttonPin);

  if (estadoAtualBotao != ultimoEstadoBotao) {
    momentoUltimaMudancaBotao = millis();
  }

  if ((millis() - momentoUltimaMudancaBotao) >=
duracaoDebounce) {

    if (estadoAtualBotao != estadoConfirmadoBotao) {
      estadoConfirmadoBotao = estadoAtualBotao;

      if (estadoConfirmadoBotao == LOW) {
        if (estadoAtualSemaforo == estadoVerde) {
          if (botaoPressionadoNesteCiclo == false) {
            botaoPressionadoNesteCiclo = true;
            Serial.println("Botao registrado para este
ciclo.");
```

```
          }
        }
      }
    }
  }

  ultimoEstadoBotao = estadoAtualBotao;
}

void lerUltrassonico() {

  if (millis() - momentoUltimaLeituraUltrassonico <
duracaoLeituraUltrassonico) {
    return;
  }

  momentoUltimaLeituraUltrassonico = millis();

  digitalWrite(triggerPin, LOW);
  delayMicroseconds(2);

  digitalWrite(triggerPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(triggerPin, LOW);

  duracaoPulsoEcho = pulseIn(echoPin, HIGH, 30000);

  distanciaCM = duracaoPulsoEcho * 0.034 / 2;

  if (distanciaCM > 0 && distanciaCM <= distanciaLimiteCM) {
    presencaDetectada = true;
  } else {
    presencaDetectada = false;
  }

  Serial.print("Distancia: ");
  Serial.print(distanciaCM);
  Serial.print(" cm | Presenca: ");
  Serial.println(presencaDetectada);
}

void controlarSemaforo() {

  if (estadoAtualSemaforo == estadoVerde) {

    if (millis() - momentoInicioEstado >= duracaoVerde) {

      if (botaoPressionadoNesteCiclo == true &&
presencaDetectada == true) {
        estadoAtualSemaforo = estadoAmarelo;
```

```
        momentoInicioEstado = millis();
        ligarAmarelo();

      } else {
        momentoInicioEstado = millis();
        botaoPressionadoNesteCiclo = false;
      }
    }
  }

  else if (estadoAtualSemaforo == estadoAmarelo) {

    if (millis() - momentoInicioEstado >= duracaoAmarelo) {
      estadoAtualSemaforo = estadoVermelho;
      momentoInicioEstado = millis();
      ligarVermelho();
    }
  }

  else if (estadoAtualSemaforo == estadoVermelho) {

    if (millis() - momentoInicioEstado >= duracaoVermelho) {
      estadoAtualSemaforo = estadoVerde;
      momentoInicioEstado = millis();
      botaoPressionadoNesteCiclo = false;
      ligarVerde();
    }
  }
}

void mostrarContagemRegressiva() {

  if (estadoAtualSemaforo == estadoVerde ||
estadoAtualSemaforo == estadoVermelho){

    if (millis() - momentoUltimaAtualizacaoLCD >=
duracaoAtualizacaoLCD) {
      momentoUltimaAtualizacaoLCD = millis();

      long tempoRestante;

      if (estadoAtualSemaforo == estadoVerde) {
        tempoRestante = duracaoVerde - (millis() -
momentoInicioEstado);
      } else {
        tempoRestante = duracaoVermelho - (millis() -
momentoInicioEstado);
      }

      int segundosRestantes = tempoRestante / 1000;
```

```
      if (segundosRestantes < 0) {
        segundosRestantes = 0;
      }

      myLcd.setCursor(0, 1);
      myLcd.print("Tempo: ");
      myLcd.print(segundosRestantes);
      myLcd.print(" ");
    }

  }

}

void ligarVerde() {
  digitalWrite(greenLed, HIGH);
  digitalWrite(yellowLed, LOW);
  digitalWrite(redLed, LOW);

  myLcd.clear();
  myLcd.setCursor(0, 0);
  myLcd.print("ATUAL: VEICULOS");
}

void ligarAmarelo() {
  digitalWrite(greenLed, LOW);
  digitalWrite(yellowLed, HIGH);
  digitalWrite(redLed, LOW);

  myLcd.clear();
  myLcd.setCursor(0, 0);
  myLcd.print("ATENCAO");
}

void ligarVermelho() {
  digitalWrite(greenLed, LOW);
  digitalWrite(yellowLed, LOW);
  digitalWrite(redLed, HIGH);

  myLcd.clear();
  myLcd.setCursor(0, 0);
  myLcd.print("ATUAL: PEDESTRES");
}
```

Segue o link da simulação do protótipo realizada pelo site WOKWI: https://wokwi.com/projects/466000506428782593

7.2 Organização Geral do Firmware

O firmware do sistema foi separado em 4 funções principais, buscando dividir as responsabilidades e realizar verificações separadas, sendo elas: lerBotao(), lerUltrassonico(), controlarSemaforo() e mostrarContagemRegressiva().

A função lerBotao() verifica se o botão já foi pressionado no ciclo atual, garantindo que só seja acionado uma vez por ciclo. Se ainda não foi pressionado define a variável botaoPressionadoNesteCiclo como verdadeira (true).

Por sua vez, a função lerUltrassonico() realiza a verificação da distância e se estiver dentro da distância limite definida (15cm atualmente) irá definir a variável presencaDetectada como verdadeira (true).

Já a função controlarSemáforo() realiza toda a lógica de troca de estados dos LEDs com base nas variáveis de controle do botão e presença pelo ultrassônico. Como já explicado no Diagrama de Estados, o fluxo segue a lógica de todo fim de ciclo do estado verde, verificar se há botão pressionado e presença detectada no ciclo atual, fazendo com que os estados consigam alternar para o amarelo, vermelho e voltar para o verde.

Por fim, a função mostrarContagemRegressiva() atualiza as contagens regressivas nos estados verde e vermelho.

7.3 Integração entre Hardware e Software

A integração entre os componentes físicos (hardware) e a lógica de controle computacional (software) foi realizada de forma centralizada no microcontrolador Arduino Uno. Essa integração foi estruturada através dos seguintes mecanismos:

- Sinais de Entrada (Hardware para Software):
    - Botoeira: O botão físico foi conectado ao pino digital 2 (buttonPin) utilizando a configuração de resistor de pull-up interno do microcontrolador (INPUT_PULLUP). Fisicamente, o pino permanece em nível lógico alto (HIGH) e é levado a nível lógico baixo (LOW) quando o pedestre pressiona a botoeira. No software, a função lerBotao() monitora essa transição e armazena o evento na variável booleana botaoPressionadoNesteCiclo.

- Sincronização do Sensor Ultrassônico: A leitura do sensor HC-SR04 exige uma integração bastante rigorosa. Através da função lerUltrassonico(), o software manipula o hardware enviando um pulso digital de 10 microssegundos para o pino Trigger (triggerPin). Esse pulso elétrico faz o sensor emitir uma onda sonora física. O pino Echo (echoPin) recebe o retorno dessa onda e altera seu estado elétrico. O software mede esse tempo de resposta em microsegundos por meio da função pulseIn(). O dado físico recebido é convertido matematicamente no software para uma variável de distância em centímetros (distanciaCM), validando a presença física do pedestre caso o valor seja menor ou igual a 15cm (distanciaLimiteCM), alterando o estado da variável presencaDetectada.

- Sinais de Saída (Software para Hardware):
  - Controle do Módulo Semáforo: O controle de estados do software gerencia a troca dos pinos digitais 4, 5 e 6 (greenLed, yellowLed e redLed) por meio do comando digitalWrite(). As funções de atuação física (ligarVerde(), ligarAmarelo() e ligarVermelho()) permitem que seja aceso ou apagado fisicamente os elementos luminosos do semáforo.
  - Comunicação de Dados via I2C: A exibição de mensagens e contagens regressivas no display LCD 16x2 utiliza o protocolo de comunicação I2C. O software utiliza a biblioteca LiquidCrystal_I2C.h para apresentar textos e variáveis numéricas, sendo tudo exibido no display físico.

7.3 Principais Decisões Técnicas

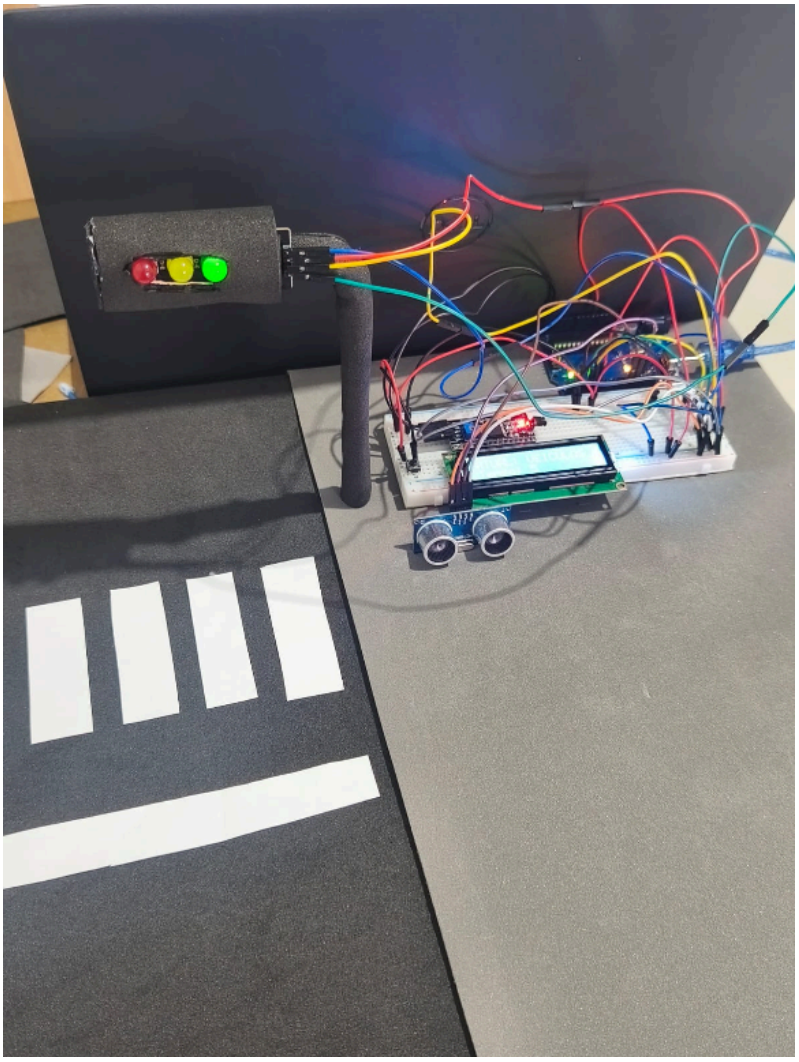
Durante o desenvolvimento do projeto, ocorreram duas decisões principais: uso de millis() no lugar de delay() e troca do sensor PIR pelo Ultrassônico.

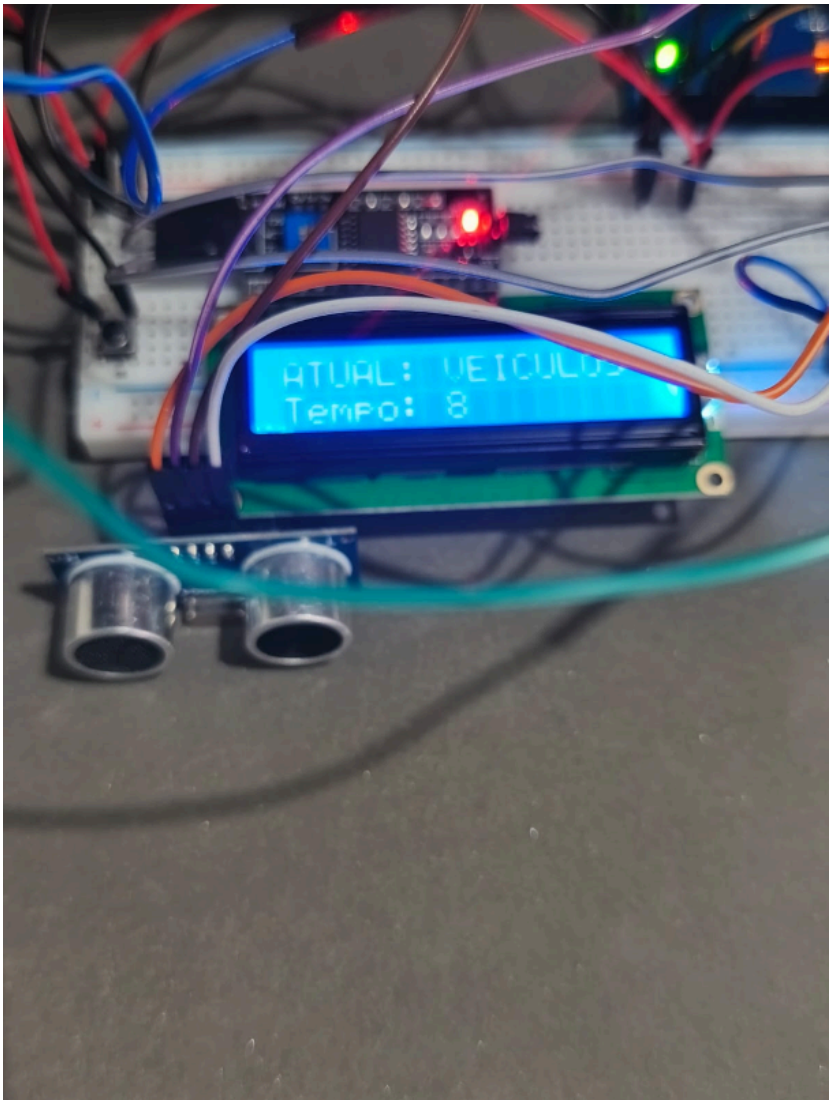
Foi decidido utilizar millis() pois é uma função que atua como temporizador que não bloqueia o fluxo. Isso significa que ao usar millis() o programa vai apenas verificar se o tempo definido já foi atingido, onde se ainda não foi, ignora o bloco de código e, se foi, executa. Essa decisão foi essencial para não travar o fluxo por exemplo quando o botão for pressionado durante o estado verde que possui contagem regressiva de 10 segundos. Assim, o botão é lido, mas se fosse delay() o acionamento do botão não seria contabilizado.

Já em relação ao uso do sensor Ultrassônico, como já foi abordado anteriormente, decidiu-se utilizar este para substituir o PIR. Essa decisão foi tomada buscando ter maior controle da distância limite utilizada e evitar falhas na leitura, como ocorria com o PIR, visto que se o elemento ficasse parado, o PIR parava de identificar como presença, o que seria um problema em termos do nosso projeto.

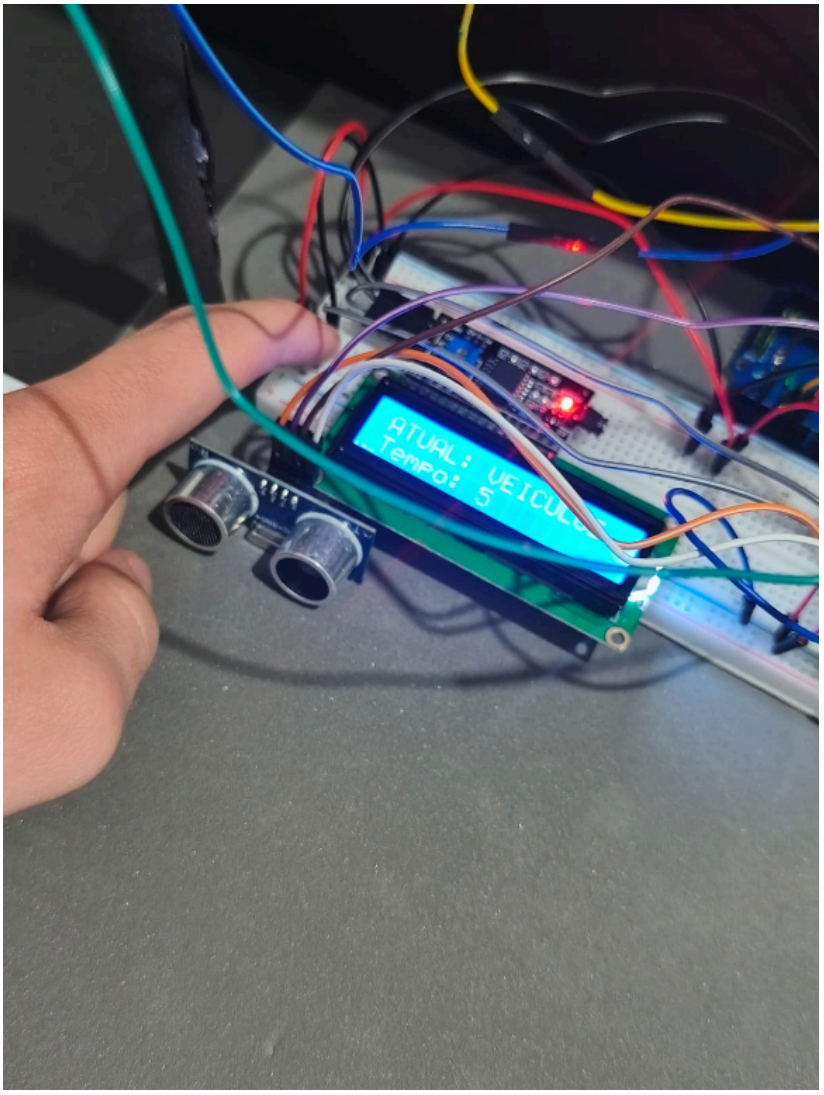
8 TESTES

8.1 Sem interação, permanecer verde

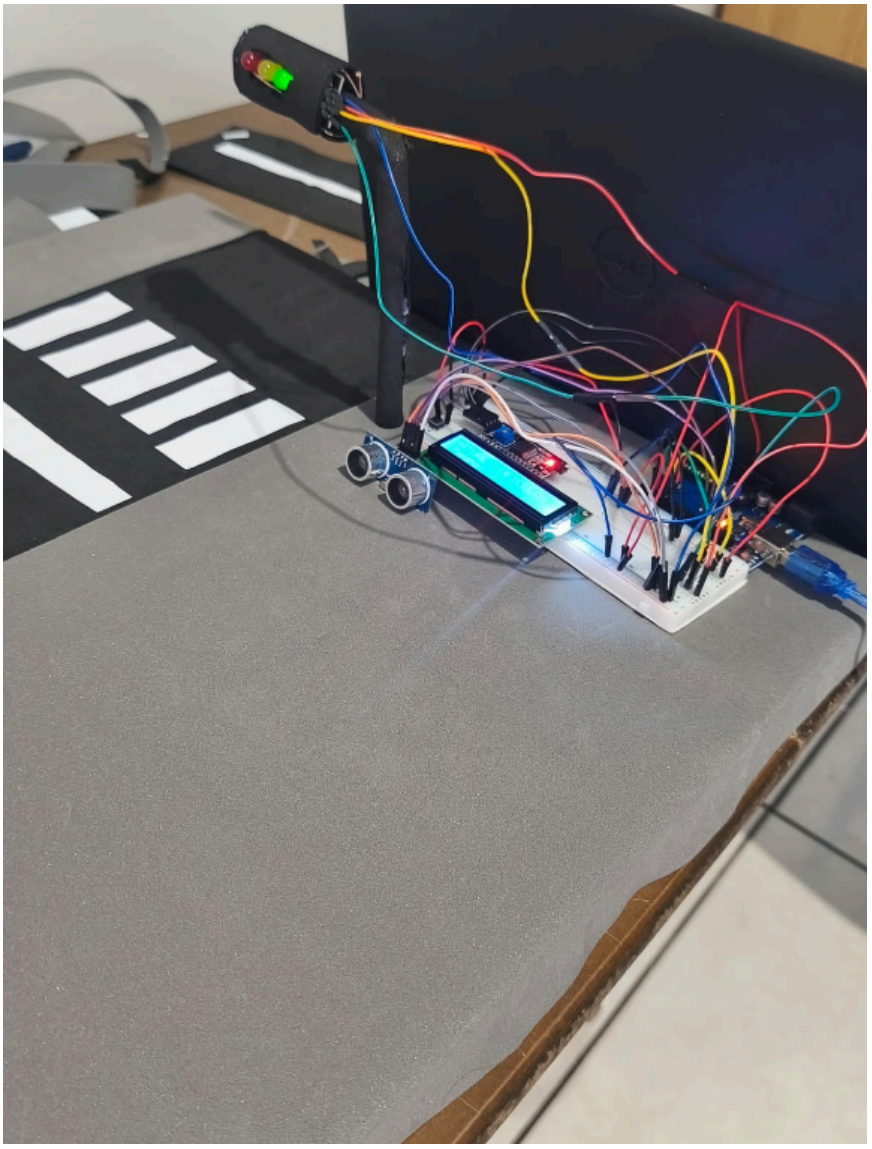


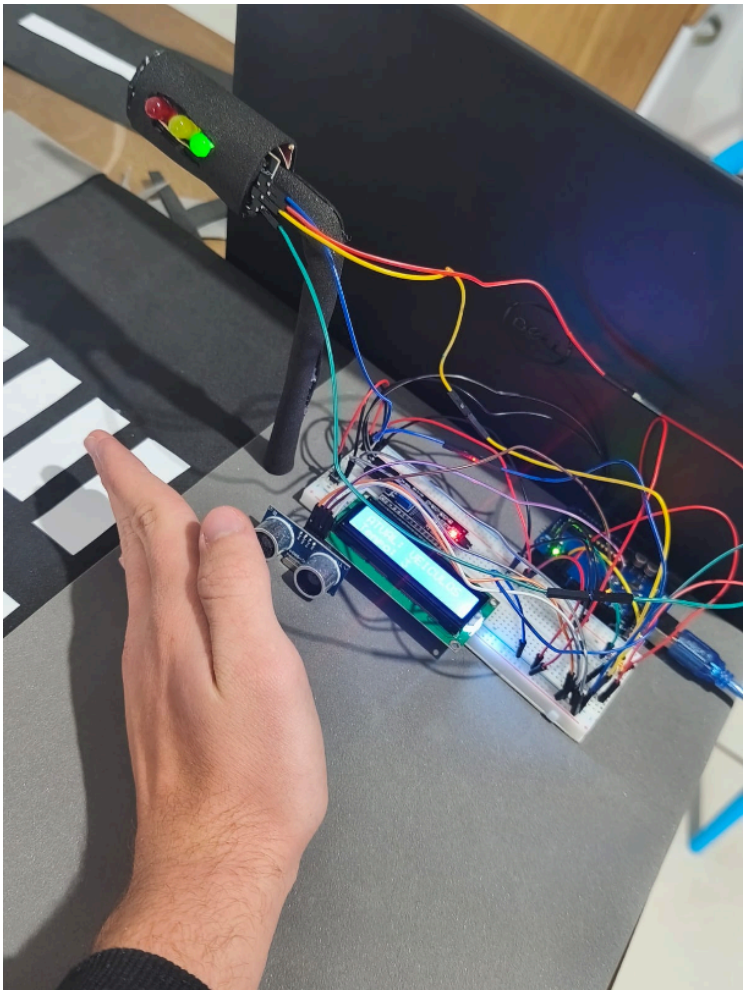


8.2 Botão acionado, sem presença, permanecer verde

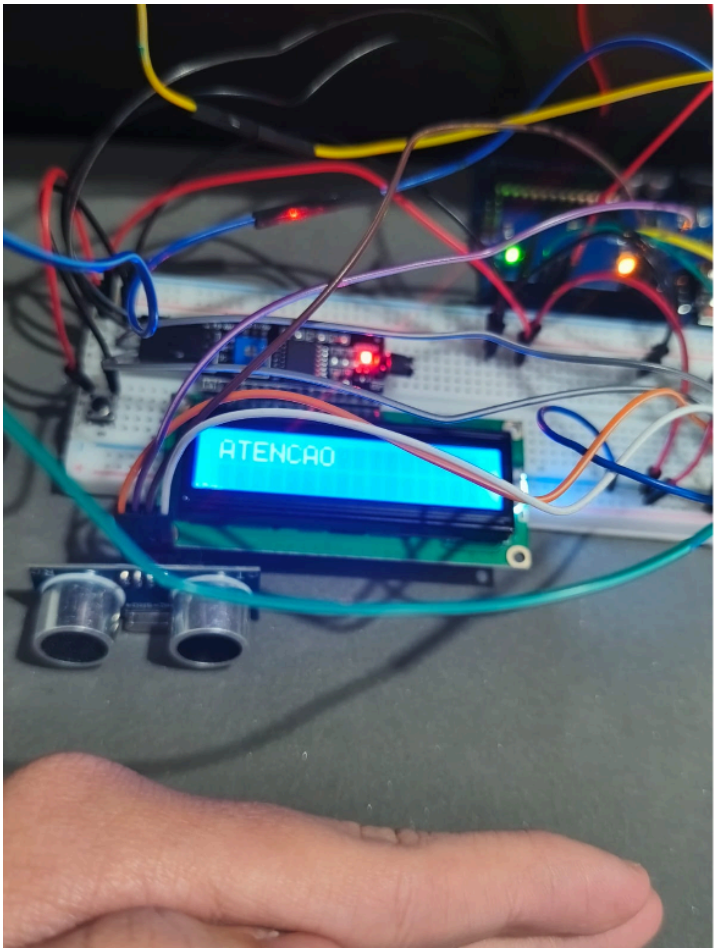


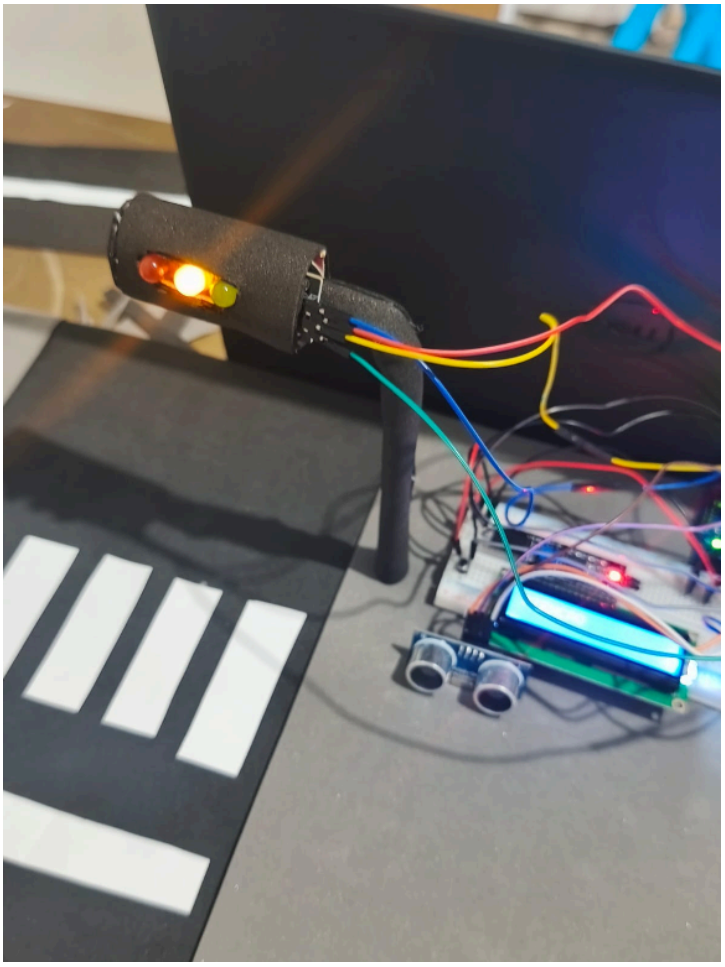
8.3 Com presença, botão não acionado, permanecer verde

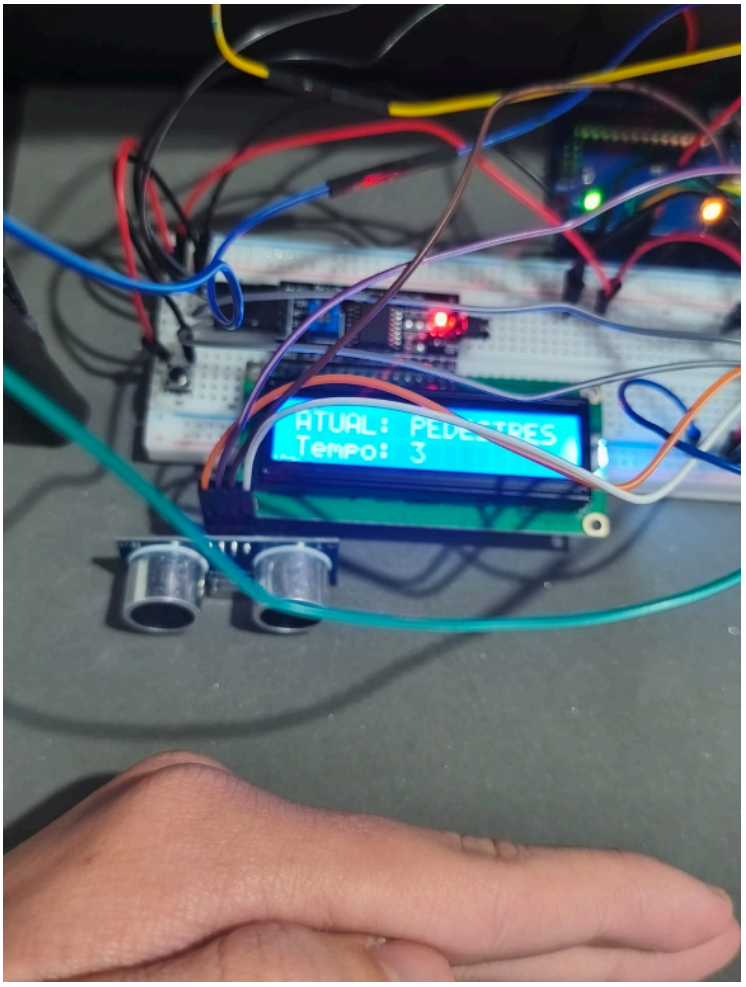


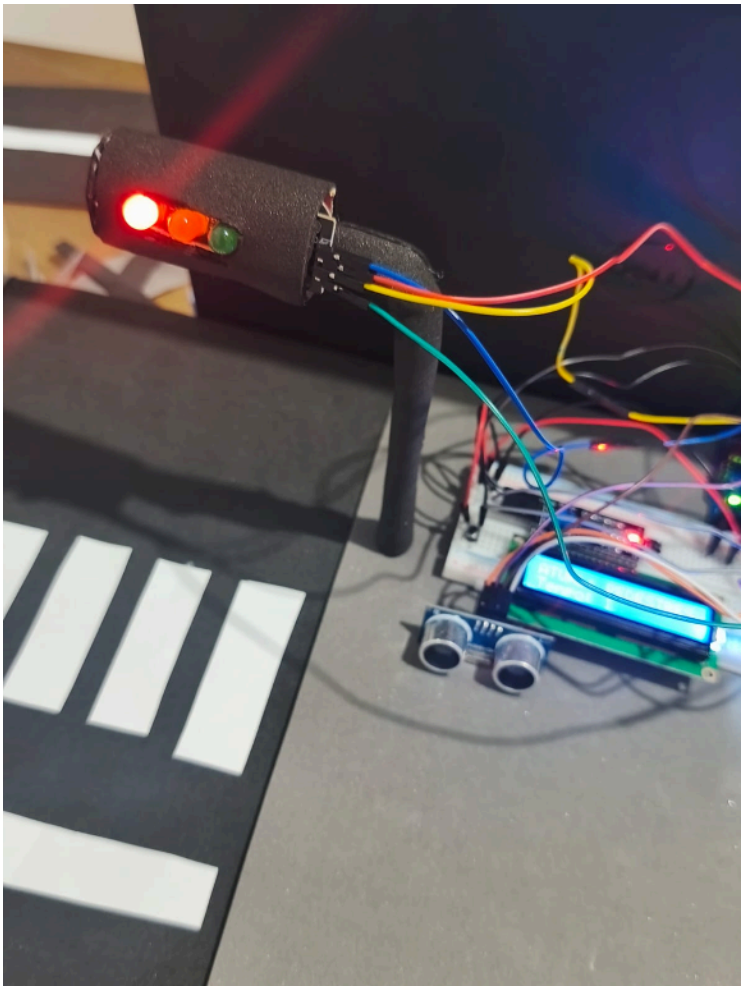


8.4 Transição para amarelo e vermelho


ATENCAO




ATUAL:
Tempo: 3



9 LIMITAÇÕES E TRABALHOS FUTUROS

Apesar do sucesso na validação do conceito, a transição da etapa de requisitos teóricos para a implementação prática revelou lacunas que devem ser abordadas em questões futuras do projeto:

- **Leitura final do ultrassônico:** no protótipo atual, a presença sempre é validada apenas ao fim de cada ciclo de 10 segundos. Uma limitação provável seria a pessoa sair nos últimos instantes do raio de identificação de presença, mesmo desejando atravessar, o que faria com que não fosse identificado presença e a pessoa não pudesse passar.
- **Foco em Acessibilidade:** atualmente, o projeto não conta com elementos focados em acessibilidade. No entanto duas melhorias poderiam ser implementadas em versões posteriores do projeto: buzzer para localização da botoeira e uso de Módulo RFID MFRC522. O buzzer poderia ser utilizado para pessoas que possuem dificuldades

visuais localizarem a botoeira de forma mais rápida e fácil. Já o módulo RFID poderia ser utilizado para aumentar o tempo de travessia dos pedestres, focado em pessoas com dificuldades físicas.

.

10 CONCLUSÃO

A execução do projeto permitiu o desenvolvimento e a validação de um protótipo funcional de semáforo inteligente baseado na plataforma Arduino Uno, integrando conceitos de sistemas ciberfísicos e cidades inteligentes para responder aos desafios contemporâneos de mobilidade urbana no cenário brasileiro. Assim, o protótipo cumpriu com êxito seu papel didático e científico, provando teoricamente que soluções de baixo custo e baseadas em lógica computacional inteligente são perfeitamente viáveis para otimizar o fluxo de tráfego, reduzir congestionamentos e promover a segurança viária nas cidades. O sistema demonstrou alta eficácia em sua lógica central. A substituição do sensor PIR pelo Sensor Ultrassônico HC-SR04 mostrou-se uma decisão técnica acertada, proporcionando leituras de distância mais precisas e estáveis para validar a presença real do pedestre na calçada. A arquitetura de software implementada sem o uso de delay(), mas sim utilizando a função millis(), garantiu que o sistema realize a confirmação do pressionamento do botão e o cálculo do sensor ultrassônico de forma correta. Além disso, a integração do display LCD 16x2 via I2C reduziu drasticamente a complexidade do cabeamento na protoboard, assegurando uma interface de comunicação visual clara e legível.

REFERÊNCIAS

BRASIL. Lei nº 12.587, de 3 de janeiro de 2012. Institui as diretrizes da Política Nacional de Mobilidade Urbana. Brasília: Presidência da República, 2012. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2011-2014/2012/lei/l12587.htm. Acesso em: 10 maio 2026.

CARVALHO, C. H. R. et al. Mobilidade urbana no Brasil: principais estudos produzidos pelo Ipea nos últimos quinze anos. Boletim Regional, Urbano e Ambiental, n. 30, p. 164-173, jul.-dez. 2023. DOI: http://dx.doi.org/10.38116/brua30art15. Disponível em: https://repositorio.ipea.gov.br/server/api/core/bitstreams/1fe9c4bb-b4c5-4f9b-8348-831955fdb9a6/content. Acesso em: 10 maio 2026.

FERREIRA, A. S. Cidades inteligentes e sustentáveis: análise e definições acerca da literatura. Revista Ibero-Americana de Ciências Ambientais, v. 12, n. 6, p. 512-521, jun. 2021. DOI: http://doi.org/10.6008/CBPC2179-6858.2021.006.0042. Disponível em: https://www.sustenere.inf.br/index.php/rica/article/view/5645/2998. Acesso em: 10 maio 2026.

MAKERHERO. Como utilizar o display LCD 16x2 no Arduino? MakerHero, 16 mar. 2022. Atualizado em: 30 nov. 2023. Disponível em: https://www.makerhero.com/blog/como-utilizar-o-display-lcd-16x2/. Acesso em: 10 maio 2026.

TOMTOM. TomTom Traffic Index 2025. TomTom, 2025. Disponível em: https://www.tomtom.com/traffic-index/. Acesso em: 10 maio 2026.

STA ELETRÔNICA. Sensor ultrassônico HC-SR04. Disponível em: https://www.sta-eletronica.com.br/artigos/arduinos/sensor-ultrassonico-hc-sr04. Acesso em: 22 jun. 2026.

DECLARAÇÃO DO USO DE INTELIGÊNCIA ARTIFICIAL GENERATIVA

Durante a preparação deste trabalho acadêmico, os autores utilizaram o Claude Sonnet 4.6 (Anthropic, 2026), o ChatGPT GPT-4o (OpenAI, 2026) e o Gemini (Google, 2026) para auxílio na estruturação de conteúdo, revisão gramatical, formatação e estilística de trechos do documento. Em todos os casos, o conteúdo foi revisado, editado e validado pelos autores, que assumem total responsabilidade pelas ideias, argumentações e decisões técnicas presentes neste documento.